sak-guard — Research Handoff and State Summary

Paused 2026-11-06 · `Pantronux-Research/sak-guard` @ `c6bc1e8`

Pantronux Research — Irfan Achmad Firmansyah

---

sak-guard — Research Handoff and State Summary

Prepared: 2026-11-06 **Status:** PAUSED at owner's direction — work resumes on request **Repo:** https://github.com/Pantronux-Research/sak-guard (public, `main` @ `c6bc1e8`) **Local tree:** `/home/kuro/recon/sak-guard/`

This document exists so that context lost between sessions does not have to be reconstructed. It records what the project *is*, what is *done*, what is *pending*, and the specific facts needed to resume without re-deriving them.

---

1. What the project is

`sak-guard` — a static pre-commit security guardrail for the Python client and agent layer of Solana applications.

The positioning that matters: every existing Solana security tool observes the wrong layer. On-chain linters and scanners (Trail of Bits `solana-lints`, Soleker) inspect contracts and bytecode. Runtime firewalls inspect a transaction at submission. **Neither sees the Python code where an agent sources a keypair, misconfigured a commitment level, or passed unvalidated model output into a transaction sink.** `sak-guard` operates there, at commit time.

It is framed as the **static complement to a runtime transaction firewall** — not a competitor. The firewall decides whether a transaction may execute; `sak-guard` decides whether the code that builds it should be merged.

Applicant / entity (for the grant):

- Name: Irfan Achmad Firmansyah
- Entity: Pantronux Research (Solo Builder)
- Region: Indonesia
- Wallet: `Ca6GSvPdnkKX3zSbqt6aPVzkmLXNoBaaP62hxPY3RbLd`
- Contact: `Pantronux-Research/sak-guard`

---

2. Technical state

The engine

sak_guard/ — `analyzer.py`, `strategies.py`, `cli.py`, `solana.py`, `__init__.py`. Pure stdlib, no runtime dependencies, installable as a pre-commit hook that does not touch the caller's environment.

Nine patterns in three families:

twidthwidthID	Family	Detects

SOL-1 (a–d)	Literal	Key base58/seed literals, 12/24-word mnemonics, `b58decode` literals bound to key-like names, key names interpolated into f-strings
SOL-2a	misuse	send/submission with no commitment and no confirmation
SOL-2b		with `simulate_transaction` result whose `.err` is never checked
SOL-2c		with blockhash → intervening work → send (temporal, heuristic)

[illegible]	[illegible]	[illegible]
SOL-3 (a–d)	artefact provenance	model output reaching `Pubkey()`, `transfer`, `send_transaction`, `send_raw_transaction`

Every pattern emits `found` / `clear` / `n/a`. Non-Solana input returns `n/a` rather than `clear`, so a clean generic-Python file is never mistaken for a clean Solana file.

Verified state

```
tests        : 36 passed, exit 0
corpus       : 32 real files, 5,670 lines (sendaifun/solana-agent-kit-py @ main)
findings     : 11       precision: 73% (8 true / 3 false)
trajectory   : 41% -> 57% -> 73%
cli contract : --fail-on all levels correct; stdin is '-' (not --stdin)
               exit 0 clean / 1 findings / 2 usage-or-IO
packaging    : build + twine check PASS; License-Expression: Apache-2.0
               (PEP 639, no License:: classifier); wheel has no caches or tests
ci           : publish.yml on release:[published], id-token scoped to publish job
               ci.yml matrix 3.10-3.13 == requires-python >=3.10
downstream   : pre-commit installs the repo; vulnerable -> 1, clean -> 0
```

Per-pattern precision: PS-SOL-1 and PS-SOL-3 produced **zero false positives** across 5,670 lines. PS-SOL-2c is the weak pattern at **3/5** — both errors come from a blockhash fetched on the line immediately before the send still counting the send itself as intervening work. It is retained at `medium` severity deliberately, and its measured failure is published.

Defects found in our own tool by the evaluation

Fixed: slash-in-literal read as division (now AST-gated on `ast.BinOp(ast.Div)`); `f"{kp.pubkey()}"` read as a key leak; division by a module constant; duplicate emission; `range(max_retries)` flagged as an off-by-one; a nonexistent path exiting **0** (silent success — the worst pre-commit failure mode); `prog="sec-audit"` instead of `sak-guard`.

---

3. The paper

`paper/paper.tex` — **TechRxiv preprint**, `\{}documentclass[conference]{IEEEtran}`, self-contained (BibTeX embedded in a `filecontents*` block).

Title: *AST-Driven Static Analysis and Taint Tracking for Autonomous Agent Execution Layers in Decentralized Systems: The `sak-guard` Pre-Commit Guardrail for Solana Agent Kit*

Sections: I Introduction · II Related Work and Theoretical Framing · III Threat Model and Taxonomy · IV Detection Engine Architecture · V Empirical Evaluation · VI Limitations and Threats to Validity · VII Conclusion and Future Work

Terminology established: PS-SOL-3 is framed as **inference artefact provenance**; the dataflow rules are a** provenance verification gate**; the sink set is the **transaction layer's trust boundary**. The artefacts tracked are **heterogeneous inference artefacts** (a completion, a tool parameter extracted from it, a `json.loads` subscript).

Compilation (verified, Docker `texlive/texlive`):

```
7 pages · 27 bibitems · 0 undefined citations · 0 undefined references
0 overfull hbox · 11 underfull · recency 24/27 = 89% (target >=80%)
```

Build: `paper/build.sh` — clears generated files first, falls back to Docker when `pdflatex` is absent. **Never build by hand on a tree that already contains a `references.bib`** (see §6).

Future-work direction anchored: a shared specification layer — write defect families once as an artefact schema and express rules as OWL/SHACL constraints, checkable statically, against runtime traces, and over post-incident evidence. This is the stated **doctoral direction**: inference-artefact integrity and provenance for autonomous agent systems.

4. Bibliography state

27 entries. Every one resolved through CrossRef or OpenAlex — **none written from memory**. arXiv DOIs live at DataCite, not CrossRef.

Paywalled (8): `firmansyah2024forensic`, `yang2024falsepositives`, `borlakova2026comparative`, `liu2025bidirectional`, `kuhne2022multilevel`, `ulain2024featureselection`, `zhao2025soleker`, `yang2024sokai`.

Reading audit: 27 cited · 19 obtainable · 16 read in full or at abstract level · 3 partial · 8 metadata only. See `paper/CITATION_AUDIT.tex`.

Errors found by reading (fixed in `c6bc1e8`):

1. `zou2023universal` listed 4 authors — the paper has **6** (Carlini, Nasr missing).
2. `yang2024falsepositives` listed 7 — **two did not exist** (Jin, Chen) and one real

author (Sun) was missing. Corrected to 5.

1. `hou2024llmse` was cited for a thesis it does not advance (it is an SLR of LLMs

across SE). Claim rewritten.

Known issue to resolve: the paper cites `ulain2024featureselection` (CCIS 2024, feature *selection*). The stronger and more accurate citation is her **IEEE Computational Intelligence Magazine 19(3):26–41, 2024** paper (interpretable feature *vectors*) — DOI `10.1109/mci.2024.3401342`, Xplore id needed, paywalled at Xplore; free TechRxiv preprint 403s to automated fetch. Shortlist written up in `paper/UL_AIN_CITATION_SHORTLIST.md`.

5. Pending work

Blocked on owner action

1. **PyPI Trusted Publisher (OIDC)** — must be linked from the PyPI dashboard,

naming workflow `publish.yml`, environment `pypi`, repo `Pantronux-Research/sak-guard`. Until then `publish.yml` cannot mint its token.

1. **Grant submission** — `SAK_GUARD_GRANT_APPLICATION.md` is complete

(**$4,000 / 2 milestones**: M1 $1,500 packaging, M2 $2,500 Solana pattern bank). Not submitted.

1. **Five accidental Superteam agent accounts** (`schema-probe-*`, zero

submissions, zero funds) — owner decided to leave them dormant. No self-service removal exists.

Papers to retrieve (ranked by impact if read)

[illegible]	Work	DOI / ID	Why it matters

twidth **Sukker** 10.1109/ase63991.2025.00209,
ASE Xplore
2025 **11334576** lated

work; its abstract confirms it analyses contract *bytecode*, which is what makes our client-layer gap real —

but full text unread

twidth **Ali** 10.1109/mci.2024.3401342 stronger
Ain
IEEE
CIM
2024

citation than the CCIS paper currently used

twidth **Yang** 10.1109/saner60148.2024.00075 invoked
et al.
SANER
2024

as methodological precedent

twitKühne10.1007/s10270-021-00951-5
SoSyMThe
2022 doc-
toral
di-
rec-
tion

twitttwitterdiakova,
risklu,as
yangfield24sokai,
cao,con-
gantext

Technical, not blocking

- PS-SOL-2c precision (3/5) — a genuine improvement target; the failure mode is

identified precisely (adjacent-line firing).

- No recall claim is made: no ground-truth labels exist for the corpus. A

multi-repository corpus with adjudicated labels is required before any recall figure would be defensible.

---

6. Traps learned (do not re-learn these)

- **`filecontents*` does NOT overwrite an existing `references.bib`.** A stale

copy silently shadows the embedded block and `bibtex` drops whatever changed in it **with no warning**. The `filecontents` package cannot fix it — it is obsolete and defers to the kernel. Use `build.sh`.

- **Zero undefined citations ≠ clean compile.** Citations and cross-references are

separate counters. A `\{}ref` to an unlabelled equation renders as `??` while the citation count stays 0. Check both.

- **Verify the effect, not the write.** A green local check cannot distinguish a

pushed commit from one that never left the machine. Scrub-token → push is one atomic sequence: set token URL → push → scrub → **verify from the remote**.

- **`gitignore` does not untrack.** `git add -A` before writing `.gitignore`

commits bytecode; use `git rm -r -cached`.

- **A path that does not exist must not exit 0.** An exit code derived from

findings *present* returns success when a path yields no files — a typo silently passes the hook. Assert the exit code for inputs that **must fail**.

- **Metadata verification ≠ having read the paper.** Bibliographic facts can be

perfectly resolved while the claim attached to them is unsupported, and author lists can be wrong in both directions. Read the abstract at minimum.

- **Never fabricate a citation field.** A DOI, venue or year that could not be

resolved is marked for author confirmation, never guessed.

---

7. Where everything lives

```
/home/kuro/recon/sak-guard/           release tree (public repo)
  sak_guard/{analyzer,strategies,cli,solana,__init__}.py
  tests/{test_cli.py,test_solana_patterns.py,fixtures/}
  docs/{SOLANA_PATTERN_BANK_SPEC.md,M2_VERIFICATION_REPORT.md}
  paper/{paper.tex,build.sh,CITATION_AUDIT.tex,UL_AIN_CITATION_SHORTLIST.md,README.md}
  .github/workflows/{publish.yml,ci.yml}
  .pre-commit-hooks.yaml  pyproject.toml  README.md  SECURITY.md  LICENSE

/home/kuro/recon/SAK_GUARD_GRANT_APPLICATION.md    ($4k / 2 milestones)
/home/kuro/recon/SOLANA_RECON_DOSSIER.md
/home/kuro/recon/ACCIDENTAL_AGENT_ACCOUNTS.md
```

`/home/kuro/recon/fpcheck/real/`	`32-file evaluation corpus`

Commits of record: `c6bc1e8` (citation corrections) · `c6bc1e8`-1 `8fbcb11` (II-A recalibration + VUW citations) · `6c9b06a` (verified DOI + build.sh) · `05919a3` (UAT fixes) · `ba9eba3` (base-bank fixes, precision 73%) · `280ccc5` (M2 pattern bank) · `v0.1.0-alpha` tag.

One-line resume: the tool works and is measured; the paper is submission-ready except for four paywalled papers worth reading and the PyPI publisher link; the grant is written and unsubmitted.